

《编译器构造实验》Lab1 实验报告

后缀表达式 (Postfix)

学校	中山大学计算机学院
课程	编译器构造实验 (DCS292)
姓名	梁力航
学号	23336128
日期	2026-03-26

目录

1. 实验目标
2. 实验环境
3. 输入语言定义
4. 设计与实现
5. 编译与运行
6. 运行日志示例
7. 样例测试与结果
8. 单元测试（加分项）
9. 性能比较（加分项）
10. 文档生成
11. 实验结论
12. 实验心得与反思
13. 附录

1. 实验目标

在原始实验软装置基础上，实现可提交、可复现、可测试的后缀表达式转换程序，满足评分项中功能、规范性与鲁棒性要求。

- 1. 将 `Parser.lookahead` 从静态成员改为实例成员。
- 2. 消除 `rest()` 尾递归，改为循环实现。
- 3. 扩展错误处理：错误分类、错误定位、错误信息输出，并支持 `--recover`。
- 4. 补全 Javadoc 文档注释并生成 HTML 文档。
- 5. 完成 JUnit 单元测试与递归/迭代性能比较。

2. 实验环境

项目	配置
开发语言	Java
JDK 要求	JDK 1.5+
本机验证环境	OpenJDK 11.0.24 LTS
执行平台	macOS (zsh) / Windows (cmd)
测试框架	JUnit 4.13.2 + Hamcrest 1.3

3. 输入语言定义

输入为一行中缀表达式，仅允许数字 `0~9` 与运算符 `+`、`-`，不允许空白字符。

```
Expr -> Term (('+'|'-') Term)*
Term -> digit
digit -> '0'..'9'
```

程序输出对应后缀表达式；当输入有误时，`stdout` 追加 `(error)`，并在 `stderr` 输出详细错误信息。

4. 设计与实现

解析方式 循环状态机	lookahead 实例字段	错误类别 词法 + 语法	恢复模式 <code>--recover</code>
---------------	-------------------	-----------------	--------------------------------

4.1 尾递归消除

原程序通过 `rest()` 递归处理后续运算。新版本以 `expect0operand` 和 `pending0p` 维护状态，使用循环完成等价逻辑，减少递归调用开销。

4.2 错误分类与定位

类别	触发条件	示例	输出形式
词法错误	空白字符、非法字符	1 +2 / 1a+2	LEXICAL error at position n
语法错误	缺运算符、缺运算量、空表达式	95+2 / 9-5+-2	SYNTAX error at position n

5. 编译与运行

macOS / Linux

```
./build.sh
./run.sh
./run.sh --recover
./run.sh --quiet < testcases/tc-001.infix
```

Windows cmd

```
build.bat
run.bat
run.bat --recover
run.bat --quiet < testcases\tc-001.infix
```

6. 运行日志示例

6.1 正确输入：9-5+2

```
$ printf '9-5+2\n' | java -cp bin Postfix
Input an infix expression and output its postfix notation:
95-2+
End of program.
```

6.2 语法错误：95+2（缺少运算符）

```
$ printf '95+2\n' | java -cp bin Postfix
Input an infix expression and output its postfix notation:
9 (error)
End of program.
SYNTAX error at position 2: missing operator
```

6.3 词法错误：1 +2（空白字符）

```
$ printf '1 +2\n' | java -cp bin Postfix
Input an infix expression and output its postfix notation:
1 (error)
End of program.
LEXICAL error at position 2: whitespace not allowed
```

7. 样例测试与结果

用例	输入	期望输出	实际输出	结论
tc-001	9-5+2	95-2+	95-2+	通过
tc-002	1-2+3-4+5-6+7-8+9-0	12-3+4-5+6-7+8-9+0-	12-3+4-5+6-7+8-9+0-	通过
tc-003	95+2	9 (error)	9 (error)	通过
tc-004	9-5+-2	95- (error)	95- (error)	通过

8. 单元测试（加分项）

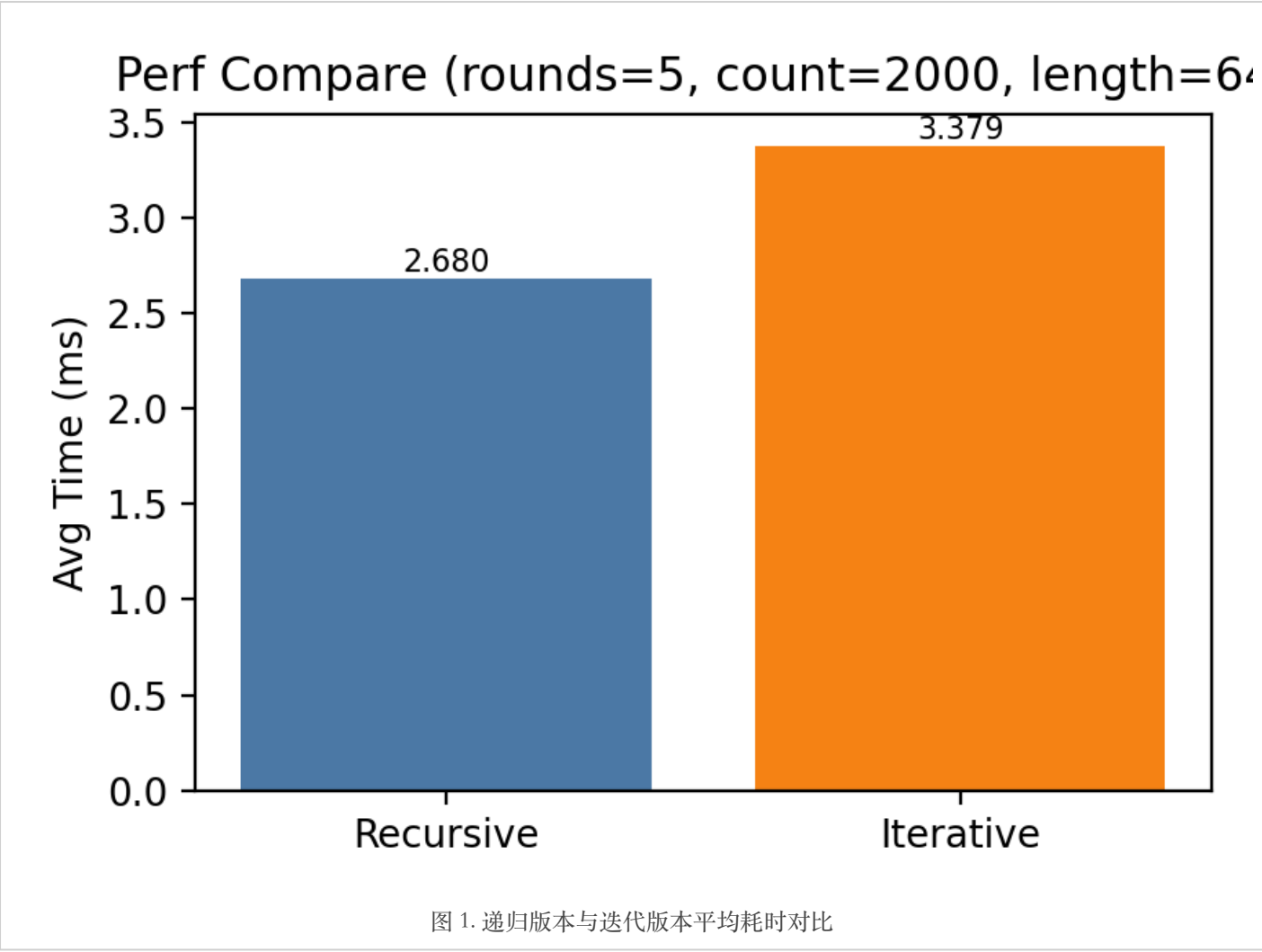
测试总数	通过数	失败数	覆盖范围
6	6	0	正常+异常

```
$ ./test.sh
Running JUnit tests...
JUnit version 4.13.2
.....
Time: 0.01
OK (6 tests)
```

9. 性能比较（加分项）

参数： rounds=5 , exprCount=2000 , length=64

版本	平均耗时 (ms)
Recursive	2.680
Iterative	3.379



说明：迭代版包含更完整的错误检测路径，本次测量中略慢属于合理现象。性能对比应基于功能等价与统一统计口径。

10. 文档生成

Javadoc 通过 `doc.sh / doc.bat` 生成到 `doc/`，覆盖 `Parser`、`ParseResult`、`ParseError` 等核心类。

11. 实验结论

1. 将 `lookahead` 由静态改为实例字段后，状态管理更符合面向对象设计。
2. 尾递归改循环后，控制流更直接，便于扩展与维护。
3. 错误分类与定位显著提升了程序鲁棒性和可诊断性。
4. 跨平台脚本使提交结果在 macOS 与 Windows 上均可复现。

12. 实验心得与反思

12.1 从功能正确到工程可交付

本实验最大的提升不在“能否输出后缀”，而在于把解析器、错误处理、测试、文档和脚本打通，形成可验证、可复现的完整提交物。

12.2 设计取舍

为了兼容样例输出，`stdout` 保持简洁；详细错误输出放在 `stderr`，使自动比对与问题定位互不干扰。

12.3 后续改进

- 支持括号、乘除与优先级。
- 完善错误恢复策略（同步符号/短语级恢复）。
- 接入 CI 自动执行编译、测试与文档构建。

13. 附录：脚本与依赖

文件	用途
<code>build.sh / build.bat</code>	编译源码到 <code>bin/</code>
<code>run.sh / run.bat</code>	运行主程序
<code>test.sh / test.bat</code>	执行 JUnit 与样例对拍
<code>doc.sh / doc.bat</code>	生成 Javadoc 文档
<code>perf.sh / perf.bat</code>	运行性能比较
<code>lib/</code>	JUnit 4.13.2 + Hamcrest 1.3